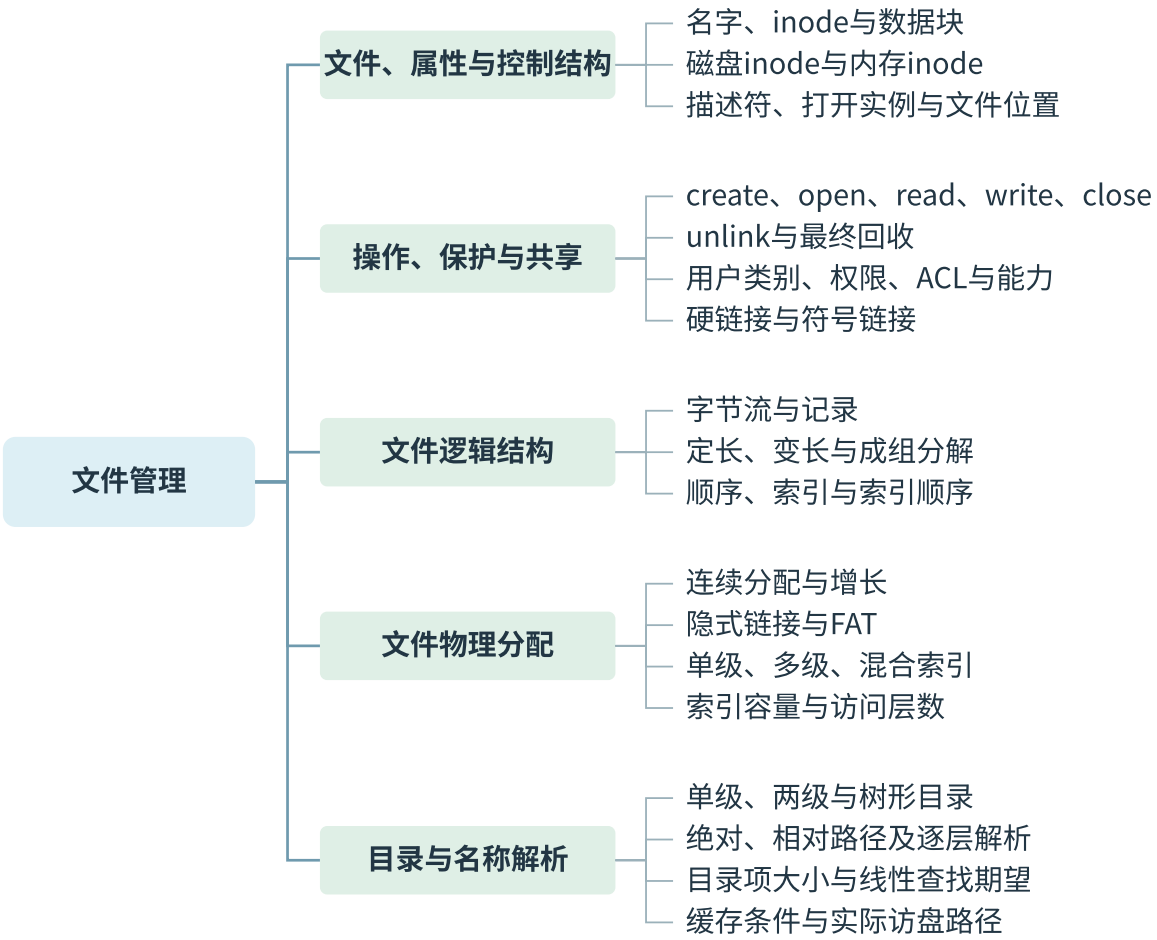
文件管理 4

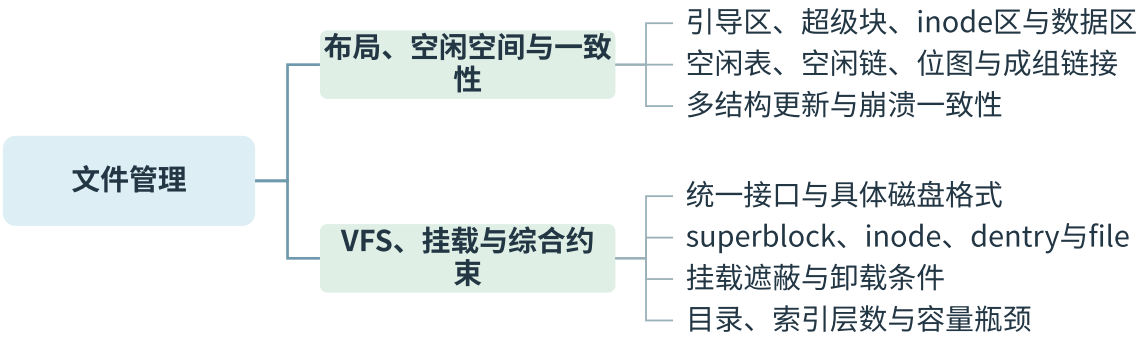
知识导图



错项排除·同号不一定同文件

inode 号要在所属文件系统内解释。两个不同分区上都出现 inode10，不足以证明它们指向同一文件；同一文件系统中，两个名字解析到同一个 inode 才表示这种对象共享。判题时先确定命名空间，再比较数字。

知识导图（续）



最小反例·关闭只撤销本次使用引用

一个文件有名字 a，进程打开后再关闭，目录里的 a 仍存在，文件数据也应保留。close 减少的是打开使用关系，unlink 取消的是名字关系。只有满足没有硬链接、没有有效使用引用等条件，才讨论对象最终回收。

4.1 沿导图画出名称路径与数据路径

文件题先画四类节点

1. 名称路径：根或当前目录，逐层目录项，以及解析出的 inode。
2. 使用关系：进程 fd、打开实例、共享 inode，标明位置指针属于哪一层。
3. 数据路径：文件内偏移到逻辑块号，再经过直接或间接索引取得物理块号。
4. 成本与容量：逐个扣除已缓存节点；分别计算数据块、索引块、inode 和空闲管理的约束。

不变量·文件身份不决定当前偏移

独立 open 同一 inode 可产生两个打开实例，分别维护位置；dup 或继承则可能共享同一个打开实例及位置。始终沿 “fd 到打开实例到 inode” 画引用箭头，就能判断哪项状态共享，而不是仅凭文件名相同推结论。

定义 4.1.1 (名称、文件本体和一次打开是三个对象)

目录项建立名字与文件标识的联系；inode 保存文件本体的属性和块定位；打开实例保存这次使用的访问方式及当前位置，进程描述符再引用打开实例。三者的共享关系、计数与生命周期分别判断。

4.1.1 混合索引容量的逐层推导

每增加一级索引，覆盖数乘一次扇出

设块长为 B 字节，每个块号占 e 字节，一个纯索引块可容纳 $q = \lfloor B/e \rfloor$ 个地址；inode 有 d 个直接项和各一个一级、二级、三级间接项。直接区覆盖 d 个数据块；一级覆盖 q 个，二级覆盖 q^2 个，三级覆盖 q^3 个。因此理论用户数据上限为 $(d + q + q^2 + q^3)B$。这要求相应地址字段可表示所需块号、所有分支可用，且未遇文件大小字段或卷容量等更紧上限；索引块开销另外计算。

□

文件系统把 “按名称找到数据” 分成几步：由路径查目录项，由目录项找到文件控制信息，再把文件内偏移转换成存储块位置。目录、索引节点、打开文件表和数据块分别承担不同工作；把它们笼统当成 “文件的信息”，就会把访问次数算混。

本章文件内字节偏移和逻辑块号从 0 开始；“第几个记录” 若采用自然序号则从 1 开始。除题目另有定义，1 KB 为 1024 B。计算访盘次数时，要逐次说明所访问的目录块、索引块或数据块中哪些尚不在内存；已缓存的内容不重复计入。

4.2 文件、文件控制块与索引节点

文件是具有名称和一定属性的持久数据集合。从使用者看，它可以是字节流，也可以由记录组成；从存储看，它最终占据若干物理块。文件的名称与内容不是同一个对象，一个文件可以通过多个名称被访问。

文件控制块 FCB 是系统描述和管理文件的数据结构，通常记录类型、大小、所有者、权限、时间和存储位置等元数据。将文件名与其他描述信息分离时，目录项可只保留 “文件名—索引节点号”，其余信息由**索引节点 inode** 保存。inode 是这种设计中的文件控制信息载体，不是文件数据块，也不只是 “指向文件的一个指针”。

索引节点号的意义局限于相应文件系统，不能只凭两个分区上的编号相同就判断是同一个文件。内存中的 inode 还可包含运行时锁、引

用计数等；它与磁盘中持久保存的 inode 相关，但字段不必完全相同。

目录本身也是文件，其中存放的是用于查找对象的目录项；普通文件的名字出现在目录中，数据内容并不因此存入目录。采用 inode 的模型还应区分三个位置：名字在目录项中，属性和块地址在 inode 中，用户内容在数据块中。

对象	记录什么	由什么找到它	何时改变或回收
目录项	当前目录内的名字及 inode 号	逐层解析路径、查找所在目录	建立、重命名、增删链接时改变；关闭文件不删除它
磁盘 inode	文件类型、大小、权限、链接数、数据块地址等持久信息	在本文件系统解释 inode 号	文件内容或属性改变时更新；须满足无链接、无有效使用引用等回收条件
内存 inode	已载入的文件信息及运行时锁、引用状态等	inode 缓存或打开对象引用	可由多个打开者共享；无打开者后是否立即释放取决于缓存模型
打开实例与进程描述符表	打开实例记录文件位置、访问方式；进程表把 fd 关联到实例	已返回的 fd 在所属进程内解释	open 建立，dup 或继承可共享实例，close 释放相应引用

因此 “知道文件名”“知道 inode 号”“inode 已在内存” 和 “数据已缓存” 依次消除了不同查找步骤。访盘题要把题设明确命中（已在内存）的对象从访问路径中扣除，不能由其中一个条件推得其他对象也都在内存。

错项排除·权限集合不能压成一个编号

每类用户可能同时有读、写等多种权限；若有 5 个独立许可就要表达其全部子集，需要 5 位。给用户类别和某一操作编码，只能描述一次选择，不能保存每一类用户各自的完整许可组合，二者信息量不同。

4.3 建立、打开、读写、关闭与删除

建立文件要分配文件控制信息，在父目录增加名字与文件标识的关联，并按规则初始化属性；空文件可以暂不分配实际数据块。**打开文件**按路径查找、检查权限，在内存建立或引用打开状态，返回文件描述符。打开不等于把整个文件内容读入内存。

一个常用抽象分为三层：进程文件描述符表把小整数 `fd` 指向打开实例；打开实例保存本次打开的状态，例如当前文件位置和访问方式；内存 inode 保存文件本身的共享属性及定位信息。不同教材有时把后两层合称系统打开文件表，本书计算前会说明各字段属于哪一层，避免因表名不同把状态混在一起。

`read(fd,buf,n)` 根据打开状态和当前偏移找到目标数据，必要时发起 I/O，向用户区返回实际读到的字节，并更新相应文件位置。`write` 可能分配新块、改变大小、把缓存标记为脏；返回成功不必表示数据已立即持久化到盘片。文件结束、短读、权限错误等应按题给语义处理。

关闭释放本次打开引用，必要时更新元数据；它不等于删除文件。**删除名字**从目录取消关联，硬链接计数随之变化；文件内容何时真正回收，还要检查是否仍有其他硬链接或有效打开引用。两类计数含义不同：磁盘硬链接数统计名字关联，打开引用数统计运行中的使用关系。

4.3.1 例 4-1　最后关闭，释放的是哪一层

例 4.3.1 (配套例题)

【配套例题】若文件 F 仅被进程 P 打开并访问，则当进程 P 关闭 F 时，文件系统需要完成的是（　）。

A. 删除目录文件中 F 的目录项
B. 释放 F 的索引节点所占的内存空间
C. 释放 F 的索引节点所占的外存空间
D. 将文件磁盘索引节点中的链接计数减 1

解析。本题按 “最后关闭时释放相应内存控制信息” 的简化模型处理，答案为 **B**。A、C 属于删除持久对象的工作，D 对应硬链接关系改变；关闭并不取消文件名。实际系统可以把暂时没有打开者的 inode 留在缓存里，缓存回收时机与本题抽象不同，不能据此把 close 与 unlink 混为一谈。

课程连接·两个索引属于不同层次

逻辑记录索引把关键字对应到记录或文件内偏移；物理索引把逻辑块号对应到磁盘块。数据结构中的查找复杂度可解释前一层，操作系统的间接索引层数解释后一层。遇到 “索引” 先写它的输入与输出对象。

4.3.2 文件位置可以共享，也可以独立

两个进程分别 open 同一文件，通常得到两个打开实例，各有自己的文件位置，但都指向同一文件的 inode。通过 dup 或进程继承共享同一打开实例时，文件位置也可能共享。因此 “同一个文件” 不决定读写位置是否相同；先看是否同一次打开的引用。

多个进程以写方式打开同一文件，不表示文件系统自动满足应用所需的完整事务互斥。要防止记录被交错修改，应使用规定的锁或其他同步方式。系统对单次写的原子性规则也不能任意推广为整个 “读—修改—写” 序列的原子性。

4.4 文件保护：用户类别与权限组合

保护回答谁能对对象执行什么操作。口令、加密和访问控制各有作用，不能相互替代：加密保护内容不被未授权读取，访问控制由系统按身份与权限限制操作；读权限、写权限、执行权限也不同于调度优先级。

访问矩阵以主体为行、对象为列，格中保存允许操作集合。按对象列存放形成访问控制表 ACL，便于查 “谁能使用这个文件”；按主体行组织授权可形成能力表一类机制，便于查 “这个主体能用哪些对象”。保护还涉及授权、撤销和检查时机；权限存进去之后，访问时仍须逐次检查。

4.4.1 例 4-2　权限可以同时成立，不能只编码一种

例 4.4.1 (配套例题)

【配套例题】某文件系统针对每个文件，把用户分成安全管理员、文件主、文件主的伙伴、其他用户 4 类；访问权限有完全控制、执行、修改、读取、写入 5 种。若 FCB 中用二进制位串表示不同类别用户对一个文件的访问权限，至少需要多少位?

A. 5　　B. 9　　C. 12　　D. 20

解析。对一类用户，五种权限可分别有或没有，需要 5 个独立二进制位。四类用户分别保存其权限组合，需要 $4 \times 5 = 20$ 位，答案为 **D**。用 2 位编码用户类别、3 位编码一种操作，只能表达 “是哪一类、做哪一种”，不能同时记录四类用户各自的权限组合。题目未给权限之间必然蕴含关系，因此不能因 “完全控制” 这个名称擅自压缩其他位。

4.5 逻辑结构：字节流和记录怎样被使用

无结构文件向应用提供字节序列，记录边界由应用自行解释；所谓无结构不是没有字节顺序。**有结构文件**由记录组成，可以定长或变长。定长记录且不跨块时，记录位置容易由长度计算；变长记录需要额外的边界、长度或索引信息。

顺序文件按记录次序存放，适合顺序处理；按关键字组织的顺序文件可利用有序性查找，但插入维护可能较贵。索引文件为记录或记录组维护索引，便于定位；索引顺序文件把顺序组织与索引结合。这些是应用看到的、或逻辑层规定的记录组织；它们与下面讲的物理块分配（连续、链接、索引）不是同一维度。

例如变长记录文件可以用一个逻辑索引定位记录所在字节范围，文件本身再通过物理索引块定位磁盘块。两个 “索引” 服务不同层次，题目问哪个对象的空间、哪个访问链，就沿对应关系计算。

错项排除·FAT 少读数据块仍需追链

FAT 驻留内存时，定位后面簇不必逐个读取前面数据块，但仍要沿内存中的链获取后继号。其优势是减少磁盘 I/O，不能直接推成按逻辑下标常数时间定位；表规模还由整个受管卷的簇数决定。

4.6 物理分配：连续、链接与索引

连续分配记录起始块与长度，逻辑块 k 直接映射为起始块加 k，顺序和随机访问都容易；但文件增长可能受相邻空间限制，空闲空间也可能有外部碎片。按区段扩展可缓解增长问题，但不是每个文件永远只有一个连续区间。

隐式链接分配在各数据块内保存后继块号，目录记录首块等信息。逻辑上连续的数据可散在任意物理块；增长容易、无需一个大连续洞，但要找后面的块通常必须沿链读取，链指针也占用空间。损坏一个指针会影响后续定位。

显式链接分配把后继关系集中到文件分配表 FAT。表若驻留内存，沿链查找不必每步读取数据块，但定位第 k 个簇仍要执行链追踪，不等于具有直接随机下标。FAT 一般为每个受管簇保存一项，规模取决于受管卷的簇数；不能仅凭某个文件的最大长度推断整个卷的 FAT 大小，除非题目另外限定了两者的关系。

索引分配为文件记录各逻辑块的物理块号，可单级、多级或混合索引。根据逻辑块号查相应索引项即可定位，支持随机访问；索引本身占空间，小文件使用大索引块时可能浪费。UNIX 式混合索引让小文件直接索引，大文件逐渐使用间接层级。

比较访盘次数时，先统一起点：逻辑块号 k 从 0 开始，目录项或 inode 已在内存，查找路径上的数据块和索引块均未缓存，也没有预存的中间块号；FAT 行另假定分配表已驻留内存。

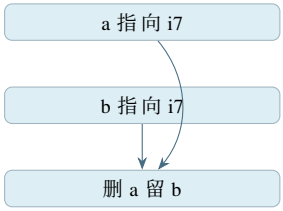
分配方式	怎样定位第 k 个逻辑块	读取目标块需访盘	额外组织空间
连续	起始块号加 k，可直接定位	1 次	起始块号、长度等描述信息
隐式链接	从首块读取后继指针，逐块追踪	$k+1$ 次	数据块中的后继指针
显式链接 FAT	在内存表中追踪 k 条链接，再读目标块	1 次，但仍有内存追链开销	全卷每簇的表项；驻留表占内存
单级、多级或混合索引	按块号确定索引项，逐级取得下层块号	经 h 级间接索引时 $h+1$ 次；直接指针为 1 次	文件的索引项和索引块

分配方式	增长与碎片
连续	扩展受相邻空间制约；可能有外部碎片
隐式链接	可离散追加；无外部碎片
显式链接 FAT	可离散追加；受空闲簇及表规模约束
单级、多级或混合索引	可离散追加；范围受各级索引容量限制

FAT 支持不读取前面数据块而访问指定块，这正是它相对隐式链接的优势；这不表示由块号定位物理块只需常数次内存访问。“无外部碎片” 也不表示最后一个数据块一定装满，各方式仍可能留下块内未用空间。题目改变 inode、FAT 或索引块的缓存条件时，先删去已命中的那一级访问，再计总数。

最小反例·删一个名字不等于删文件

同一文件 inode 为 i7，目录项 a 和 b 都指向它。删除 a 后仍有 b 这一硬链接，文件内容继续可达。把 a 叫作原文件不赋予它特殊地位；硬链接之间对称，不能说删除 “原名字” 就必然使另一个硬链接失效。



两个目录项对应同一 inode；删除名字 a 后，名字 b 仍可引用文件。

4.6.1 例 4-3　混合索引中，一次访问要经过几层

例 4.6.1 (配套例题)

【配套例题】文件索引节点中有 10 个直接索引指针，一级和二级索引指针各 1 个。磁盘块为 1 KB，每个索引指针 4 B。某文件索引节点已在内存，把该文件偏移量为 1234 和 307400（按字节编址）处所在的磁盘块读入内存，需要访问的磁盘块个数分别是（　）。

A. 1、2　　B. 1、3　　C. 2、3　　D. 2、4

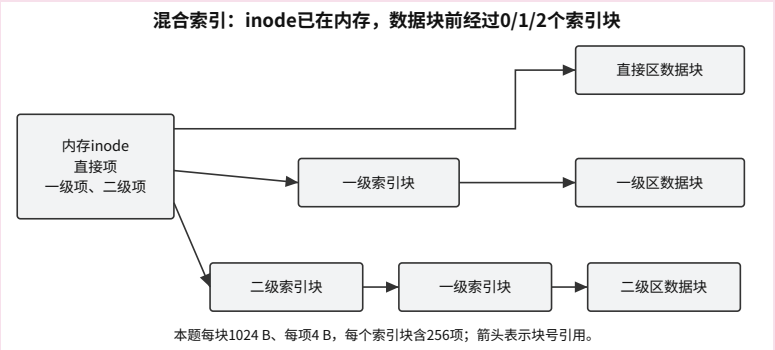


Figure 4.1: 直接、一级、二级路径经过的索引块数不同；图中只画少量代表项

解析。一个索引块能存 $1024/4 = 256$ 项。直接索引覆盖前 10 KB；一级间接再覆盖 256 KB；二级间接再覆盖 256^2 KB。

1234 小于 10×1024，属于直接区。inode 已在内存，可直接取目标块号，只读 1 个数据块。307400 大于 $(10 + 256) \times 1024 = 272384$，进入二级区；需读二级索引块、对应的一级索引块和数据块，共 3 次，答案为 **B**。

还可定位到项：$307400 = 300 \times 1024 + 200$，逻辑块号 300；减去前面的 $10 + 256 = 266$ 块，二级区内偏移 34 块，即先取二级索引的第 0 项，再取其所指索引块的第 34 项。块内偏移 200 不影响要经过的索引层数。

本题结构能表示的数据长度为 $(10 + 256 + 256^2) \times 1024$ B，即 64 MB + 266 KB；其中不把索引块计为文件用户数据长度。若索引块已经缓存，访盘次数会减少；若 inode 尚不在内存，还应计入取得它的相应访问。题目说 “索引节点已在内存”，不能再加一次 inode 读盘。

4.6.2 例 4-4　链接文件修改一条记录，要把读和写都计入

例 4.6.2 (配套例题)

〖4.2-025〗一个记录文件采用隐式链接分配，定长逻辑记录为 100 B，采用记录成组分解，盘块为 512 B。目录项已读入内存，对第 22 个逻辑记录完成修改，共启动磁盘多少次?

A. 3　　B. 4　　C. 5　　D. 6

解析。按原题记录不跨块、每块容纳 5 条记录的模型，第 1—5 条在第 1 块，依次类推，第 22 条位于第 5 块。隐式链要从第 1 块沿后继指针读到第 5 块，共读 5 次，再把修改后的第 5 块写回 1 次，总计 6 次，答案为 **D**。目录已在内存只免去目录读取，不能免去沿链的数据块读取。

盘块中的链接指针若占用部分字节，本题只要每块剩余空间仍足够容纳 5 条 100 B 记录，结论不变；若允许跨块记录、索引已缓存或采用 FAT，则必须重画访问路径。不要把 “修改第 22 条” 理解成写 22 次，也不要漏掉最后的回写。

不变量·缓存只删去对应访问

目录内容已在内存，只免去读取该目录内容；若目标 inode 和数据尚未缓存，仍要分别取到。写访盘路径时为每个节点标 “已缓存/未缓存”，再计边上的实际读取，避免一个命中条件被扩张成整条路径全命中。

4.7 目录和路径解析：逐层找到名字

单级目录把所有文件名放在一张表，简单但命名冲突多；两级目录按用户分开，缓解重名但共享和多层分类受限；树形目录让目录内再有子目录，便于按层次组织。绝对路径从根开始，相对路径从当前工作目录开始；`.` 表示当前目录，`..` 表示父目录，二者的解析都受权限与挂载规则约束。

访问 `/home/u/a.txt` 并不是在一张全局名字表里直接跳到数据。典型 inode 模型先从根目录查 `home`，取得其 inode，再查 `u`，最后在该目录中查 `a.txt` 并取得文件 inode。各层目录、inode、数据是否缓存，决定实际 I/O 数；路径分量的数量本身不是总访盘次数。

采用 “名字与 inode 分离” 可以使目录项更小，一块容纳更多名字，从而减少线性查找的目录块数。以王道书 §4.2 的目录模型为例，3200 项、每项 16 B、每块 1024 B，需要 50 个目录块。若目标等概率落在任意一项，且逐块线性查找，各块恰有相同数量目录项，则平均读取 $(1+50)/2 = 25.5$ 个目录块；随后再读 1 个 inode 块，总期望 26.5 次。用 $50/2+1 = 26$ 是 “大量目录块” 的近似，不能当作相同概率模型下的精确值。

若目录使用哈希或树索引，查找次数应按相应结构计算；若预先只读入目录的一部分，也不能把剩余查找都视为零成本。当前工作目录使常用路径可以省去重复解析共同前缀，但不取消权限检查。

4.8 硬链接、软链接与什么时候回收文件

硬链接是另一个目录项直接指向同一文件 inode；多个名字地位相同，不存在 “原名字一删，另一个就失效” 的必然关系。增加一个硬链接通常增加 inode 的硬链接计数，删除一个名字减少计数；只有计数归零，并且不再有有效使用引用等其他条件，才能回收数据和 inode。硬链接通常不能跨文件系统，因为 inode 编号必须在本文件系统解释；通常也限制用户为目录建立硬链接，以避免目录环等管理问题。

软链接或符号链接是一个特殊文件，内容保存目标路径。访问软链接时，系统再解析这个路径；目标被删或移走后，软链接本身仍可存在，

只是成为悬空链接。它可以跨文件系统，也可指向目录；相对目标路径的解释与链接所在位置有关，不能一概按调用者当前工作目录处理。

错项排除·位图长度取决于管理范围

管理一百万个块，即使当前只剩一个空闲块，位图仍须为每个块保留一位。空闲块数量影响位值分布，不缩短受管对象集合。位图内相对位置算完后再加其起始磁盘块号，不能先从受管块号减去起点。

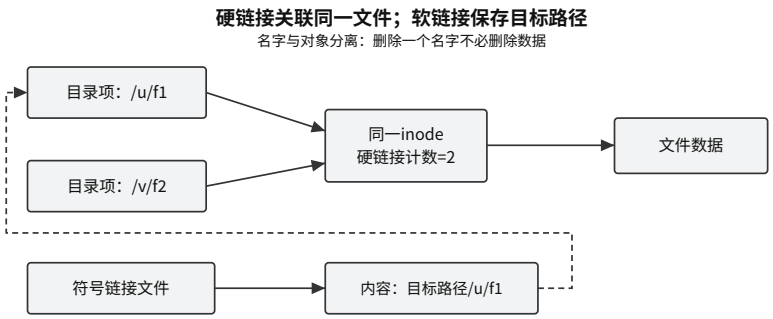


Figure 4.2: 两个硬链接指向同一 inode；软链接保存路径，经过名称解析再找到目标

4.8.1 例 4-5　同一 inode 不要求同一读写位置

例 4.8.1 (配套例题)

【配套例题】若文件 f1 的硬链接为 f2，两个进程分别打开 f1、f2，获得文件描述符 fd1、fd2，则正确的是（　）。

I. f1 和 f2 的读写指针位置保持相同

II. f1 和 f2 共享同一个内存索引节点

III. fd1 和 fd2 分别指向各自的用户打开文件表中的一项

A. 仅 III　　B. 仅 II、III　　C. 仅 I、II　　D. I、II、III

解析。 f1、f2 是同一文件的两个名字，两次打开最终引用同一个 inode，II 正确。两进程分别打开，读写位置可以独立，I 错误。每个文件描述符在相应进程的描述符表中解释，分别关联各自的打开表项，III 正确。答案为 **B**。这与共享同一个打开实例的继承或复制描述符情形不同。

4.9 文件系统的磁盘结构与内存结构

磁盘上的引导控制信息用于启动；卷控制块或超级块记录块大小、卷规模、空闲管理等总体信息；目录和 inode 区描述对象与名字；数据区保存内容。具体文件系统可采用其他布局，题目若给不同结构，应以其字段为准。超级块并不保存每个文件的索引信息。

挂载后，内存中维护已挂载文件系统信息、目录缓存、inode 缓存、系统打开对象以及进程描述符表等。缓存加快访问，但修改与写回必须遵守规定的次序和时机。一次文件扩展可能同时修改数据块、空闲位图、inode 大小和块地址；若只写了一部分就断电，会出现泄漏或结构不一致。

一致性检查可以扫描目录、inode 与块使用关系，找出不一致；日志机制先记录需要完成的更新，再按规则提交和重放，减少崩溃后恢复工作。日志不表示任意用户数据都自动获得完整事务语义，其保证取决于记录元数据还是数据、提交和写回的具体规则。这里只解释多结构更新之间的因果，不展开某种文件系统的实现细节。

4.10 空闲空间：位图、空闲链和成组链接

空闲表记录连续空闲区的起点与长度，适合按区段管理；**空闲链**把空闲块或空闲区串起来；**位图**为每个受管块保留一位，值表示空闲或占用，具体 0、1 意义由题目规定；**成组链接**把若干空闲块号存成一组，用组内的连接信息指向后续组，并在内存保留当前组的块号栈。

位图大小主要由受管总块数决定，不随当前空闲块数改变；空闲表、空闲链所需内容则与当前空闲分布相关。成组链接靠少量的组间连接把大量空闲块号组织起来，并不是把所有文件的数据链混在一起。

课程连接·统一接口可以隐藏多种布局

VFS 把查找、打开、读写等操作抽象成共同接口，再由具体文件系统实现。它和面向接口编程相似：调用约定统一，并不要求内部数据结构一致。不能因为应用都调用 read，就推断各磁盘格式具有相同的 inode 字段。

4.10.1 例 4-6　先定位位图块，再定位块内字节

例 4.10.1 (配套例题)

【配套例题】文件系统用位图表示磁盘空间分配，位图存于磁盘 32—127 号块，每块 1024 B，盘块和块内字节都从 0 编号。释放盘块号 409612 时，要修改的位所在盘块号和块内字节序号分别为（　）。

A. 81、1　　B. 81、2　　C. 82、1　　D. 82、2

解析。每个盘块的位图可表示 $1024 \times 8 = 8192$ 个数据块。$409612 = 50 \times 8192 + 12$，故目标位在位图第 50 块，即磁盘块 $32 + 50 = 82$；余数即块内第 12 位，它位于字节 $\lfloor 12/8 \rfloor = 1$，在该字节中的位序号为 4。答案为 **C**。

位图自身的起始磁盘块号 32，只在“位图内相对块号”算出以后加；不能先把受管盘块号减去 32。修改成 0 还是 1，要看位图的空闲值约定，本题只问位置，不必自行确定。

4.11 虚拟文件系统与挂载

虚拟文件系统 VFS 为不同具体文件系统提供共同的操作抽象，把打开、查找、读写等请求通过相应操作接口分派给具体实现。统一接口不表示磁盘格式也统一，更不要求所有设备保存相同的 inode 布局。

Linux 式抽象常区分超级块对象、inode 对象、目录项对象 dentry 和文件对象：其中超级块对象描述一个已挂载文件系统，inode 描述文件本体，dentry 缓存名字与父目录等关系，file 对象描述一次打开。dentry 与 inode 不是同一个英文术语；目录项对象的名称不能误写成 inode 对象。

挂载把某文件系统的根关联到现有名称空间的一个挂载点，使路径解析到该处时转入新文件系统。挂载点之下原有内容不因此被删除，只是在相应名称解析视图下被新挂载内容遮住。卸载要处理尚在使用的对象和待写回信息，不能只删一个路径字符串就认为设备已经可以安全移除。

4.12 综合例题：目录、索引与容量共同约束

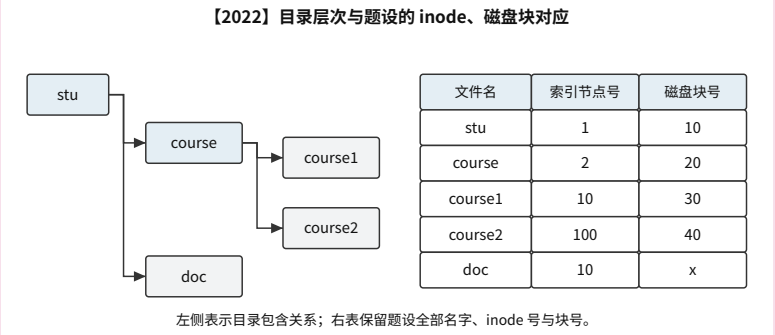
不变量·文件数量由最紧资源约束

每个图像既需要一个 inode，也需要足够数据簇。应分别计算 inode 区能容纳的个数和数据区能容纳的个数，再取较小者；若还要间接索引簇或目录空间，应按题设另计，不能只用剩余字节除文件长度。

4.12.1 例 4-7　同一索引节点怎样连接目录查找与文件扩展

例 4.12.1 (配套例题)

【配套例题】某文件系统的磁盘块大小为 4 KB，目录项由文件名和索引节点号构成，每个索引节点占 256 字节，其中包含直接地址项 10 个，一级、二级和三级间接地址项各 1 个，每个地址项占 4 字节。该文件系统中子目录 stu 的结构如下图所示，stu 包含子目录 course 和文件 doc，course 子目录包含文件 course1 和 course2。各文件的文件名、索引节点号、占用磁盘块的块号如图中表所示。请回答下列问题。



文件名	索引节点号	磁盘块号
stu	1	10
course	2	20
course1	10	30
course2	100	40
doc	10	x

Figure 4.3: stu、course 的目录层次，以及各文件的索引节点号和数据块号

（1）目录文件 stu 中每个目录项的内容是什么?

（2）文件 doc 占用的磁盘块的块号 x 的值是多少?

（3）若目录文件 course 的内容已在内存，则打开文件 course1 并将其读入内存，需要读几个磁盘块？说明理由。

（4）若文件 course2 的大小增长到 6 MB，则为了存取 course2 需要使用该文件索引节点的哪几级间接地址项？说明理由。

解析。(1) stu 的直接子项只有 course 和 doc。目录项存名字和 inode 号，不存其数据块号，故内容分别为 `(course,2)`、`(doc,10)`；course1、course2 的目录项在 course 中，不在 stu 中。图未列出 `.`、`..`，本问按图列出两个给定子项。

（2）doc 和 course1 的索引节点号同为 10，且属于同一文件系统，说明它们是同一个文件对象的两个名字。course1 的数据在 30 号磁盘块，因此 $x = 30$。不是因为名字相似而共享，也不是把 inode 号 10 直接当作数据块号。

（3）按本问给出的缓存起点，course 的目录项可在内存查到，先取得 course1 的 inode 号 10。随后读入该 inode 所在的磁盘块，从 inode 得到数据块号 30，再读 30 号数据块，共 **2 个磁盘块**。打开所需的内存表项由系统建立，不另算作磁盘块；256 B 的 inode 放在磁盘块中，读取它仍按块计。题目只说目录内容已在内存，不能顺带认定文件 inode 和数据也已经缓存；若另给这些缓存命中条件，才相应减少次数。

（4）本题按 1 MB = 1024 KB 计算，6 MB 文件需要

$$\frac{6 \times 1024\ \text{KB}}{4\ \text{KB}} = 1536\ \text{个数据块}.$$

每个间接索引块容纳 $4096/4 = 1024$ 个地址项。10 个直接项与一级间接项合计覆盖 $10+1024 = 1034$ 个数据块，仍少 $1536-1034 = 502$ 块；二级间接项可再覆盖 1024^2 个数据块。因此需用**一级和二级间接地址项**，不必用三级。这里的 1536 是用户数据块数，索引块另占空间；按先用直接区、再用各间接区的组织，一级区需要 1 个索引块，二级区需要 1 个二级索引块及其指向的 1 个一级索引块。

不变量·文件数量由最紧资源约束

每个图像既需要一个 inode，也需要足够数据簇。应分别计算 inode 区能容纳的个数和数据区能容纳的个数，再取较小者；若还要间接索引簇或目录空间，应按题设另计，不能只用剩余字节除文件长度。

4.12.2 例 4-8　先算地址覆盖，再比较 inode 与数据簇的上限

例 4.12.2 (配套例题)

【配套例题】某文件系统采用索引节点存放文件的属性和地址信息，簇大小为 4 KB。每个文件索引节点占 64 B，有 11 个地址项，其中直接地址项 8 个，一级、二级和三级间接地址项各 1 个，每个地址项长度为 4 B。请回答下列问题。

（1）该文件系统能支持的最大文件长度是多少？（给出计算表达式即可。）

（2）文件系统用 1M（$1\text{M} = 2^{20}$）个簇存放文件索引节点，用 512M 个簇存放文件数据。若一个图像文件的大小为 5600 B，则该文件系统最多能存放多少个这样的图像文件？

（3）若文件 F1 的大小为 6 KB，文件 F2 的大小为 40 KB，则该文件系统获取 F1 和 F2 最后一个簇的簇号需要的时间是否相同？为什么？

解析。（1）一个索引簇可存 $4096/4 = 1024$ 个簇号。直接、一级、二级、三级区分别覆盖 8、1024、1024^2、1024^3 个数据簇，所以最大文件长度为

$$L_{\max} = (8 + 1024 + 1024^2 + 1024^3) \times 4096 \text{ B}.$$

按二进制容量单位，这等于 4 TiB + 4 GiB + 4 MiB + 32 KiB。它只统计文件数据，不把索引簇自身算入用户文件长度。

（2）先独立计算两个容量限制。每个 inode 簇放 $4096/64 = 64$ 个 inode，故 inode 区能容纳 $2^{20} \times 64 = 2^{26} = 64\text{M}$ 个文件。5600 B 大于一个簇而小于两个簇，每个图像占 $\lceil 5600/4096 \rceil = 2$ 个数据簇，数据区可容纳 $512\text{M}/2 = 256\text{M}$ 个。两个数据簇都在 8 个直接地址项覆盖范围内，不额外占间接索引簇。因此最多存放

$$\min(64\text{M}, 256\text{M}) = 64\text{M} = 67\,108\,864 \text{ 个图像文件}.$$

这里 M 表示个数的 2^{20} 倍；数据簇尚有剩余，也不能突破 inode 的数量上限。计算采用题给的两个可用区域，不另假设目录占用或其他保留量。

（3）按本题比较的索引访问层级，时间不同。F1 占 $\lceil 6/4 \rceil = 2$ 个簇，末簇是逻辑簇 1，其簇号就在 inode 的直接项中。F2 占 $40/4 = 10$ 个簇，末簇是逻辑簇 9；前 8 个逻辑簇 0—7 用直接项，逻辑簇 8—9 进入一级间接区，所以还须沿一级间接地址读索引簇，再取其第 1 项。

在 inode 同样已在内存、相关间接索引未缓存的共同起点下，F1 的末簇号可直接从内存取得，F2 多一次索引簇读盘。若连末簇数据也要读入，则路径分别是“数据簇”与“一级索引簇 $\rightarrow$ 数据簇”，为 1 次、2 次访盘。原问只求簇号，不能把目标数据簇的读取强行加进原问；若另有索引缓存命中，实际耗时还会改变。

4.13 分支自测：沿名称与数据两条线检查

不变量·容量与索引开销分开累计

一个索引块可装 B 除以指针字节数个地址。计算最大文件长度时只累加可指向的数据块，索引块本身另占外存。若题目转问总空间消耗，才把实际使用的各级索引块加回，两个问题不能共用一个总数。

以下为原创回顾设问。

1. 关闭最后一个文件描述符，是否要求硬链接数减 1?
2. 例 4-3 中，偏移恰好为 10240 B，属于直接区还是一级间接区?
3. 某文件的硬链接数从 2 减为 1，文件内容是否回收？软链接是否增加目标的硬链接计数?
4. 管理 2^{20} 个块需要多少位图空间？若目前只有 1 个空闲块，位图是否只需 1 位?
5. 同一 inode 能否由两个打开实例保存不同的当前偏移?

4.13.1 自测答案

1. 不要求。打开引用数统计运行中的打开，硬链接数统计目录名字的关联。
2. 前 10 块覆盖偏移 0—10239，10240 是一级间接区第一个字节。
3. 仍有一个硬链接，不回收内容。软链接只保存路径，不因此增加目标硬链接数。
4. $2^{20}/8 = 2^{17}$ B，即 128 KB。位图仍为每个受管块保留一位。
5. 可以，独立打开与 inode 共享是两个不同关系。

4.14 主动回忆自测

以下为自拟题，先画对象与路径，再回答。

从名字一直追到数据与回收

1. 两个名字指向同一 inode，是否要求两个进程的当前读写位置相同?
2. close、unlink、回收 inode 与数据块分别改变哪种关系?
3. 同一文件的硬链接删去一个，另一名字是否失效？软链接为何可能悬空?
4. 4 类用户各有 5 个独立权限，用位串至少需要多少位?
5. 块长 1KiB、指针 4B、10 个直接项及一二级间接项，各区分别覆盖多少数据块?
6. inode 已在内存，相关索引与数据未缓存，读二级间接区数据块需几次访盘?
7. 位图管理 2^{20} 个盘块，需多少字节？当前只有一个空闲块会缩短位图吗?
8. 挂载新文件系统会删除挂载点下原有内容吗？VFS 是否要求各文件系统磁盘格式相同?

4.14.1 核对答案与关键条件

文件层次与容量核对

1. 不要求。独立 open 通常各有打开实例与位置；dup 或继承共享打开实例时才可能共享位置。inode 表示文件身份。
2. close 释放打开引用；unlink 取消目录名字并减少硬链接计数；最终回收还须无硬链接和有效使用引用等条件，不能混为一次操作。
3. 硬链接仍直接指向同一对象，保留名字可继续访问；软链接保存路径，目标路径失效后链接文件仍可存在。
4. 每类保存 5 个独立是非许可，合计 $4 \times 5 = 20$ 位；不是“类别编号加单个操作编号”。
5. 每个索引块 256 项，直接区 10 块，一级 256 块，二级 256^2 块；用户容量乘块长，索引块另计。
6. 二级索引块、一级索引块、数据块，共 3 次；若某一级已缓存，按实际路径减去对应读取。
7. $2^{20}/8 = 2^{17}$B，即 128KiB；管理范围未变，位图长度不变。
8. 原内容通常只是被当前名称解析视图遮住，不因此删除；VFS 统一操作接口，具体磁盘布局可以不同。

错项排除·位图长度取决于管理范围

管理一百万个块，即使当前只剩一个空闲块，位图仍须为每个块保留一位。空闲块数量影响位值分布，不缩短受管对象集合。位图内相对位置算完后再加其起始磁盘块号，不能先从受管块号减去起点。